NikPak

User Manual and Technical Guide

Version	1.13.2
Language	English (en)
Date	2026-11-14

Contents

Part I · User manual 3

1. What NikPak is 4
2. Installing and the first archive 5
3. Compression levels and speed 7
4. Passwords and encryption 8
5. Split volumes and self-extracting archives 9
6. Covers, legal statements and packages 10
7. unNik, the free browser 11
8. Skins, languages and registration 12

Part II · Technical guide 14

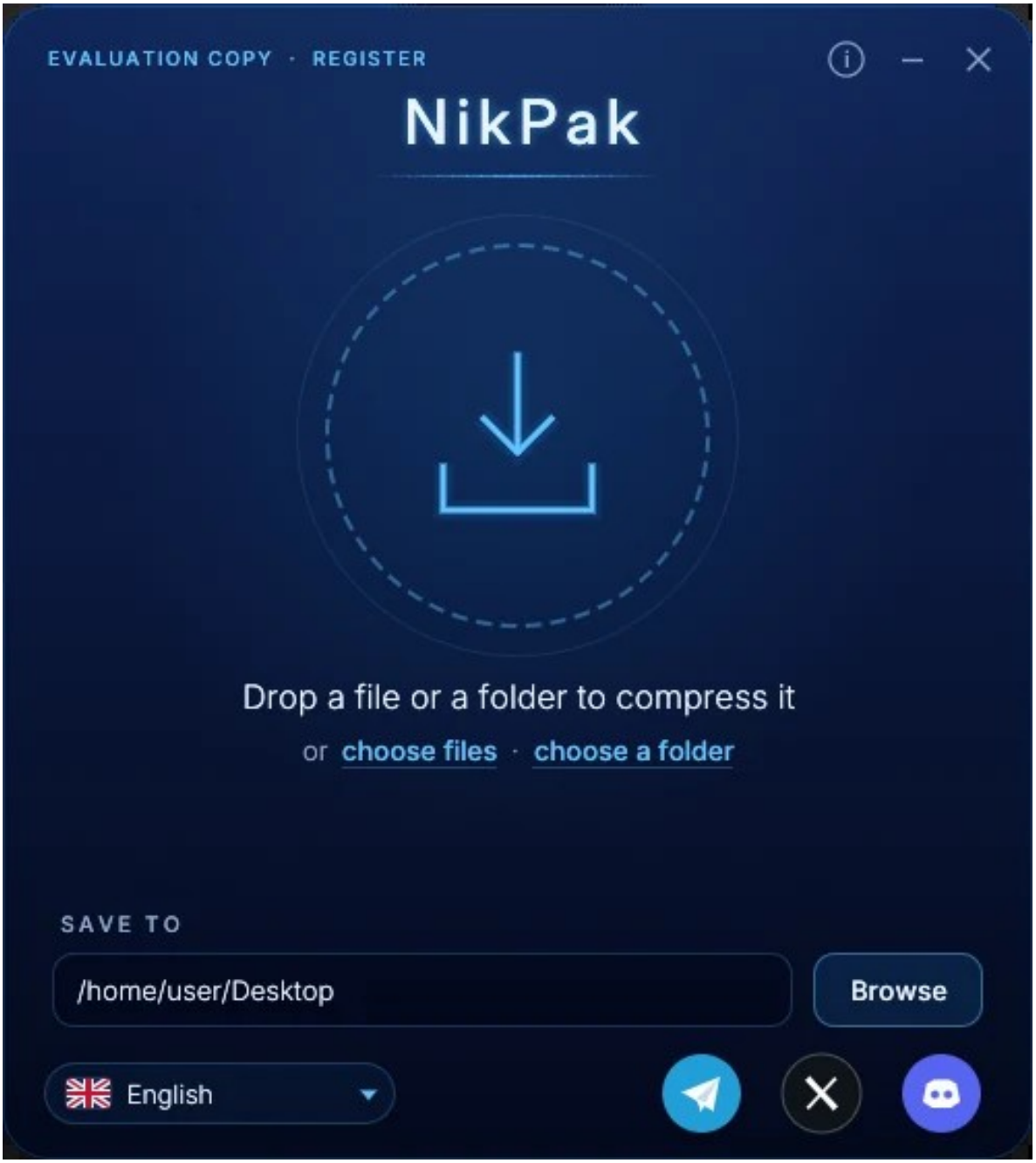
1. Architecture 15
2. The .nik format 15
3. The compression engine 15
4. Encryption, and why it is quantum-resistant 16
5. Benchmarks, hardening and building 16
6. This book 18

Part I · User manual

Everything NikEdit does, window by window.

1. What NikPak is

NikPak is a fast, encrypted archiver for the **.nik** format, and **unNik** is the free archive browser that opens those archives on any machine. Together they replace the ordinary zip in the two places it is weakest: size, and trust.



The NikPak window, waiting for a drop.

It compresses with **Zstandard**, **LZMA2** and **PPMd**, recompresses the **Deflate** inside Office files and PDFs bit-exactly, encrypts with **Argon2id + XChaCha20-Poly1305**, and wraps it in a window you can skin. Files are compressed on all cores, an archive can be split into volumes or turned into a self-extracting `.exe`, and a cover picture and a legal statement can be attached, which everyone must accept before extracting.

The two programs

- **nikpak** — the archiver: a window you drop files onto, and a command line with RAR-style syntax (`nikpak a -p secret.nik Photos\`). Shareware: free for individuals, licensed per seat for companies.
- **unNik** — the free archive browser: opens `.nik` archives in a window, browses folders, searches, views text and extracts. Free for everyone, for ever, no registration.

Both are written in Rust and run on Windows, macOS and Linux. The `.nik` format is open and documented in Part II, so an archive written today can be read by a program written tomorrow.

What it buys you

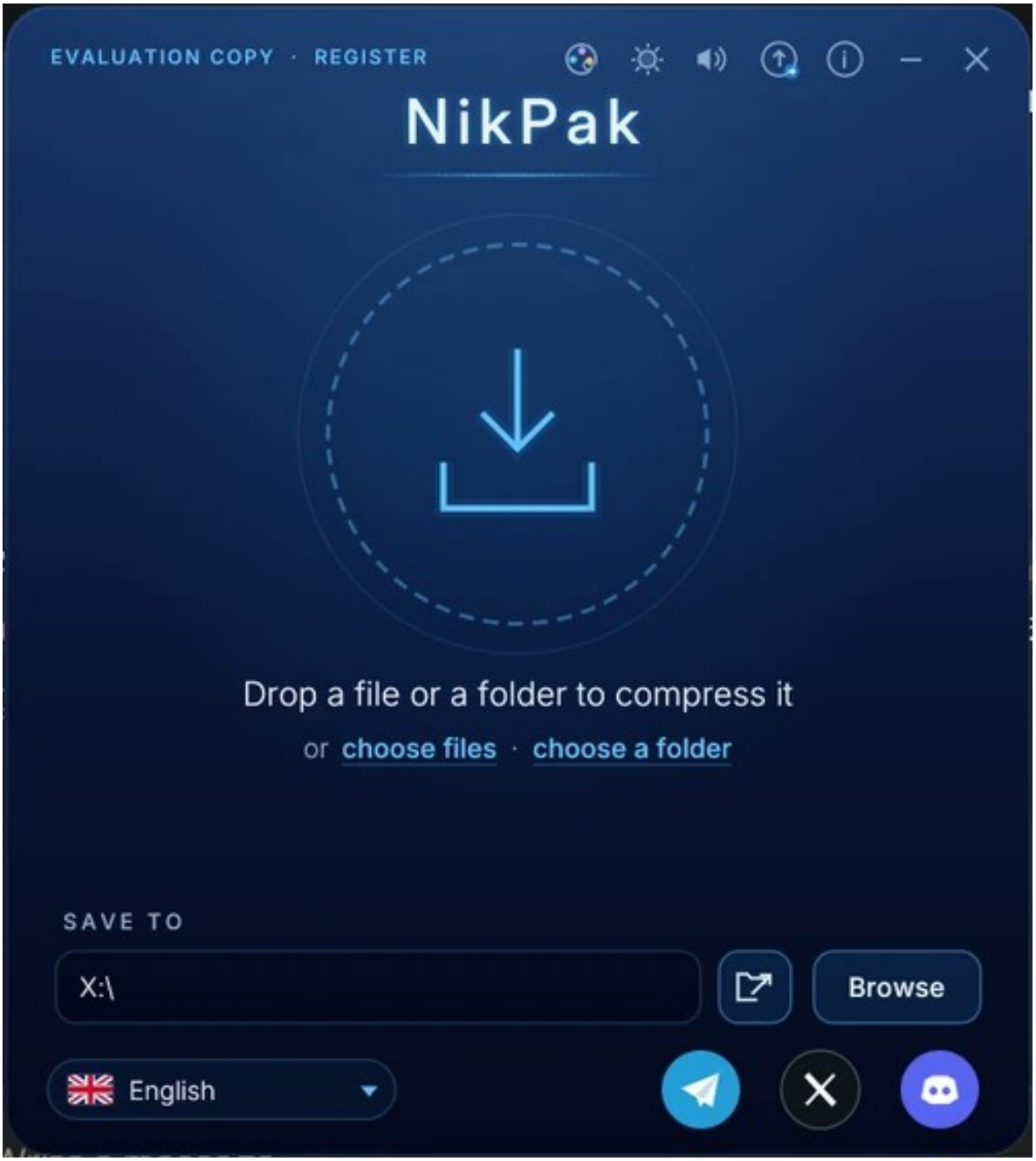
- **Smaller archives.** At the maximum level, binary data goes through LZMA2 and text through PPMd, each chosen per block — 38.8 % smaller than RAR 7 on photographs, 12–15 % on audio.
- **Real encryption.** Argon2id key derivation and XChaCha20-Poly1305; file names are encrypted too, and a wrong password or a tampered archive is detected rather than silently half-extracted.
- **Multi-core.** `-m5` splits the archive into balanced blocks and compresses each on its own thread; `-mt<N>` is the dial.
- **Lossless media filters.** WAV, AIFF, MOD and BMP are trial-compressed with purpose-built filters and the smallest result is kept.
- **Packages.** Pack files into an encrypted, sellable `.nik` package with a name, a description, a square cover and a licence to approve.

2. Installing and the first archive

Run the installer, or use the portable executable. The installer adds `nikpak` and `unNik` to the applications menu and the command line. On macOS and Linux, copy both binaries into `/usr/local/bin`.

From the window

Start NikPak and **drop files or folders on the window**. The drop box asks where to write the archive and with which settings; the archive is then built in the window, with a progress bar per file and the total. Dropping an archive does the opposite — it opens the Extract box.



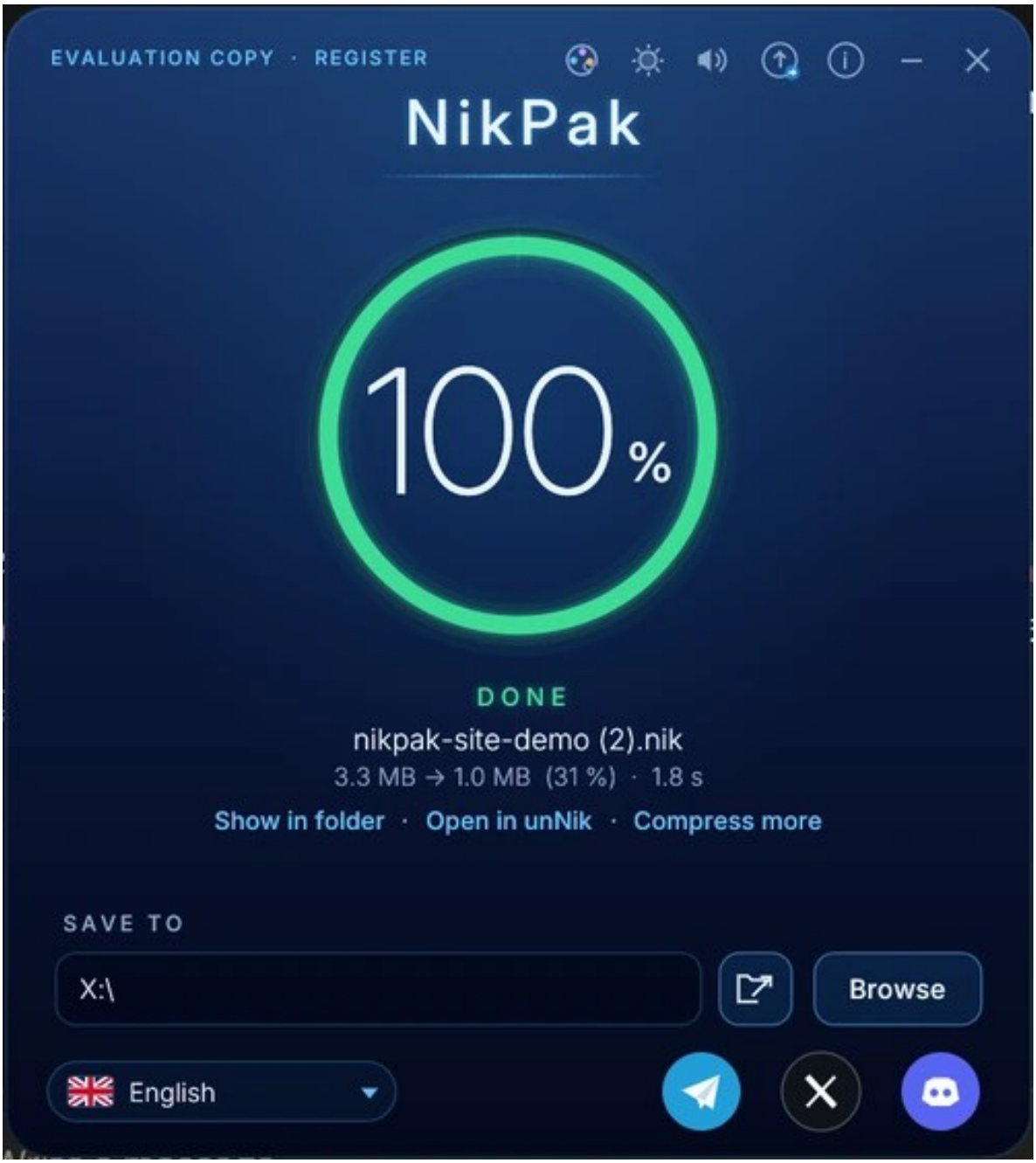
The drop box: what to pack, where to write it, and the level.

From the command line

```
nikpak a backup.nik C:\Docs\*.txt Photos\        # create an archive
nikpak a -m5 -mt8 big.nik Projects\             # maximum compression, 8 threads
nikpak a -p secret.nik Taxes2026\               # encrypted, asks for the password
nikpak x backup.nik C:\Restore\                 # extract with paths
nikpak l backup.nik                             # list
nikpak t secret.nik                             # test (asks for the password)
```

`nikpak -help` prints the whole reference, including the options this chapter does not use. A command-line-only build (no window) is available for servers.

3. Compression levels and speed



The compression box: level, threads and the estimate.

Six levels, from **Store** to **Maximum**. The default is the maximum, because an archive is usually written once and read many times — but a level is a choice, not a law, and the estimate in the box tells you what it costs before you commit.

The levels

Level	What it does	When to use it
`Store`	No compression, just the container and the encryption	Already-compressed media you want in one file
`Fastest` / `Fast`	Multi-threaded Zstandard	Huge folders, interactive work

Level	What it does	When to use it
`Medium`	Zstandard with a bigger window	The everyday choice
`High`	LZMA2 with PPMd on text	Documents, source, logs
`Maximum`	Per-block LZMA2 / PPMd, media filters, Deflate recompression	Archives you will keep and send

Threads is `-mt<N>`, or *Automatic* in the window. `-m5` already splits the archive into balanced blocks and compresses each on its own thread; the per-file work — media trials, Deflate recompression — runs in a pipeline at every level.

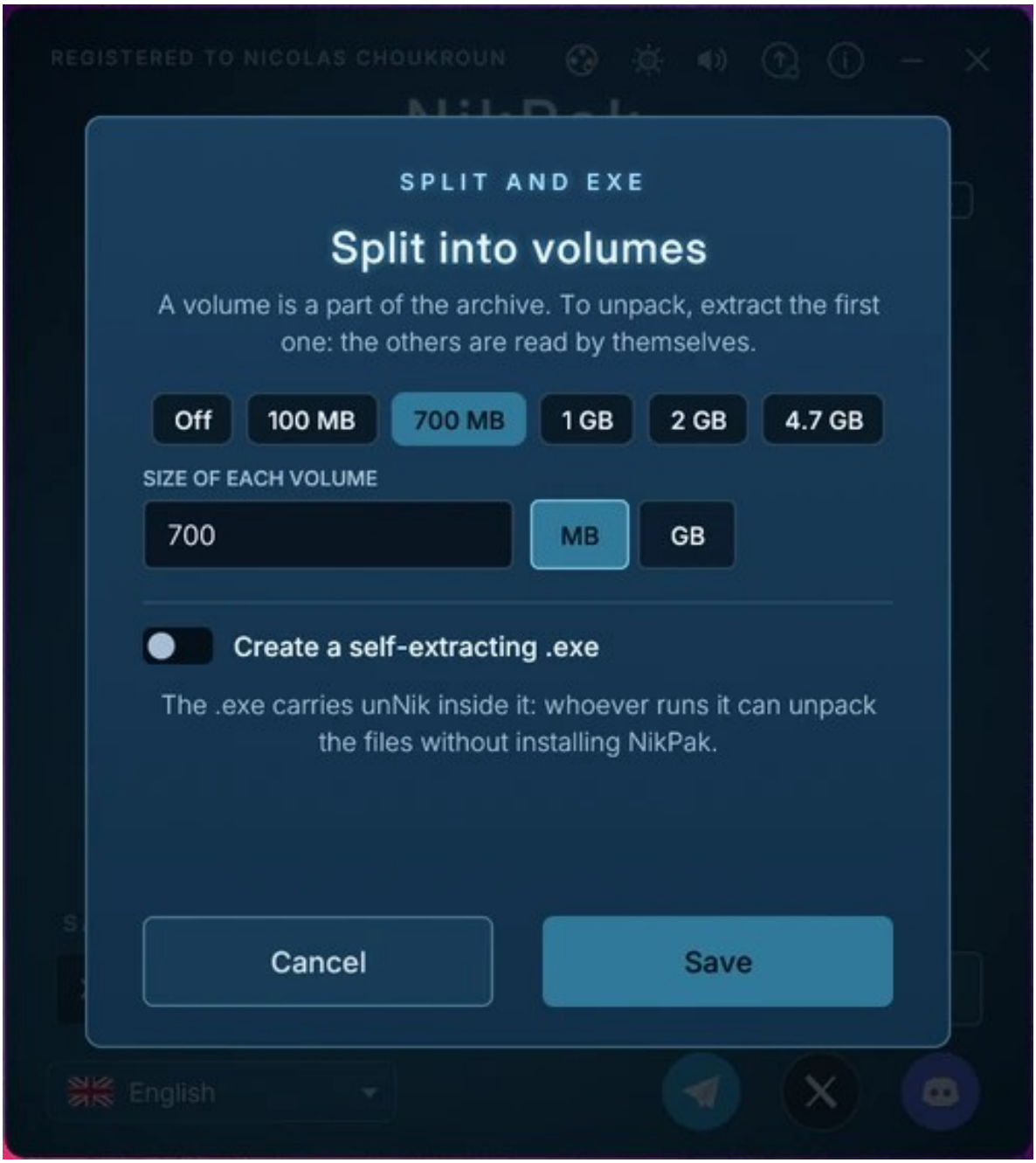
> Office documents (`docx`, `xlsx`, `pptx` …), PDFs and `.gz` files are already Deflate-compressed inside. NikPak decodes those inner streams, compresses the plain content much better, and rebuilds the original streams **bit for bit** on extraction — the archive is smaller and the file you get back is the file you put in.

4. Passwords and encryption

A password makes the archive encrypted end to end: **Argon2id** derives the key, **XChaCha20-Poly1305** encrypts the content, and the **names of the files are encrypted too**. A wrong password and a tampered archive are both detected — an archive that fails its check is reported as damaged rather than extracted into rubbish.

- `-p` asks for the password twice when creating, once when reading; it is never echoed.
- Passwords are never stored in the archive, and NikPak never keeps one between runs.
- The same password protects the split volumes and the self-extracting `.exe`.
- There is no recovery: a lost password is a lost archive, which is the point of encrypting it.

5. Split volumes and self-extracting archives



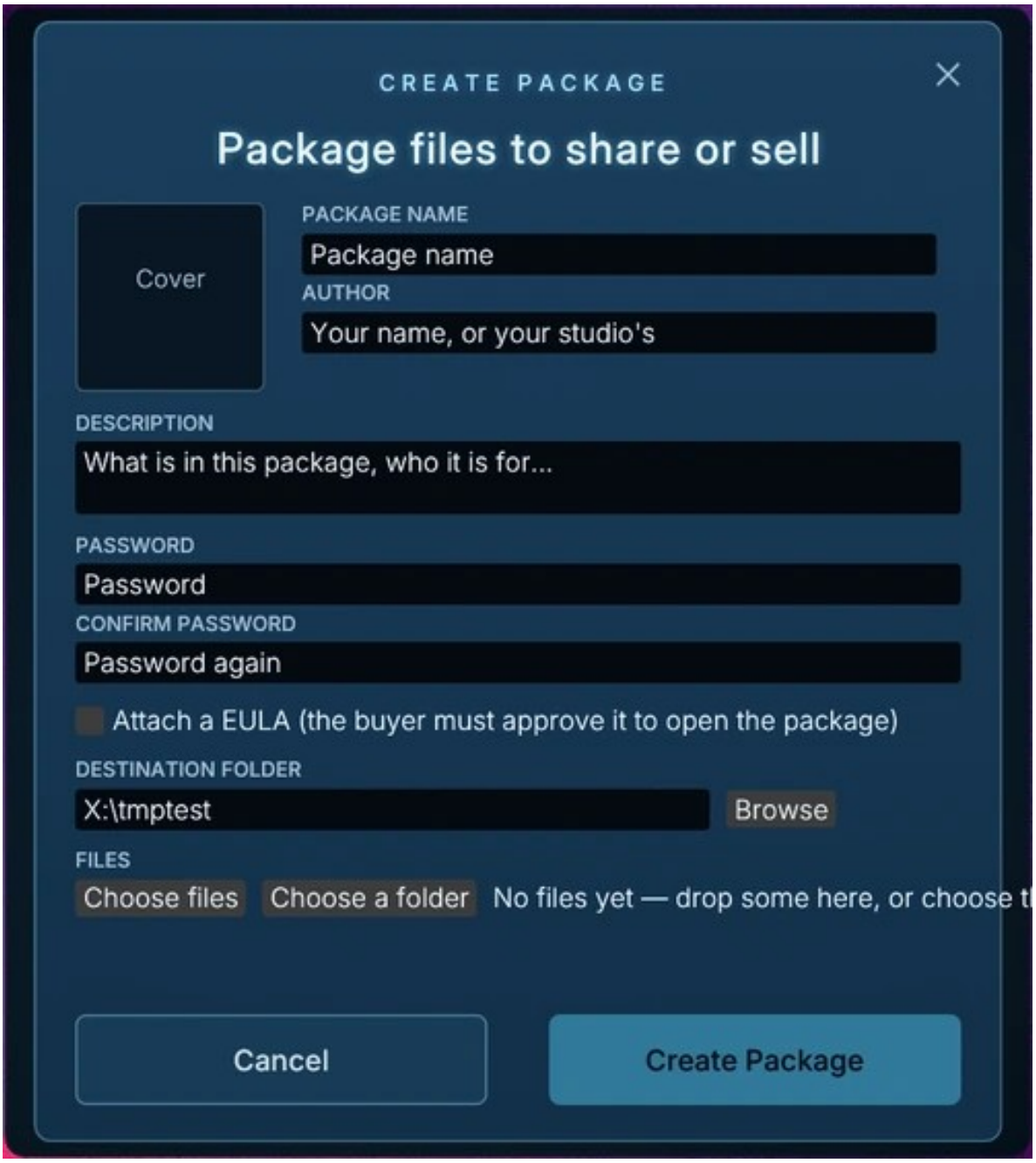
The Split box: volume sizes and the self-extracting option.

`-v` splits an archive into volumes of a chosen size — `nikpak a -v700m video.nik Movies\` writes `video.nik`, `video.n01`, `video.n02` … To extract, open the **first** volume and the rest are read automatically; the others are not opened by hand.

`-sfx` writes a **self-extracting `.exe`**: unNik with the archive appended, so a recipient unpacks it with nothing installed. It is the format to send to someone who does not have NikPak, and it can carry a password and a legal statement like any other archive.

6. Covers, legal statements and packages

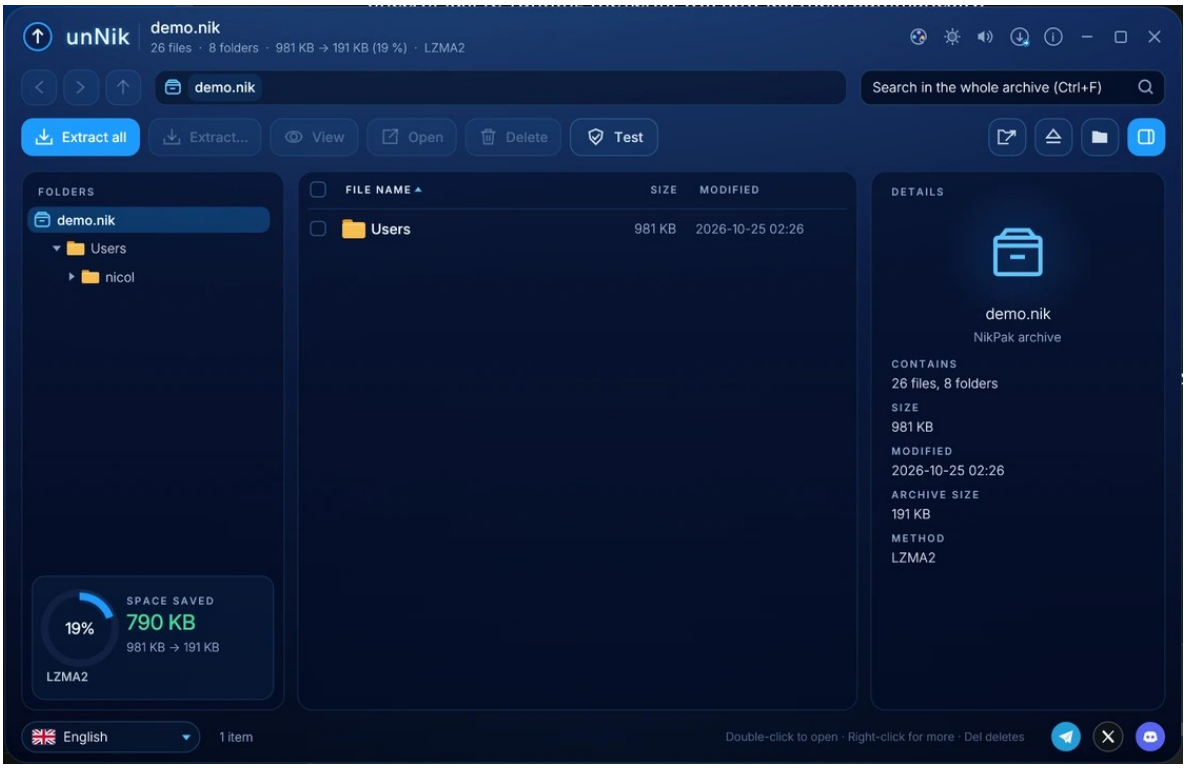
`-cover art.png` attaches a picture — square, stored at 1024 × 1024 — and `-legal terms.txt` attaches a statement. unNik shows both before it extracts anything, and nothing is read until the reader accepts. A `cover.jpg` or `terms.txt` beside the items being packed is picked up automatically.



The Create Package box: name, description, cover and licence.

Packages are the same idea taken further: a `.nik` archive with a name, a description, a square cover and a licence the recipient approves — the NikPak twin of a soundbank, made to be sold or published. *Create Package* and *Open Package* are on the file manager's menu.

7. unNik, the free browser

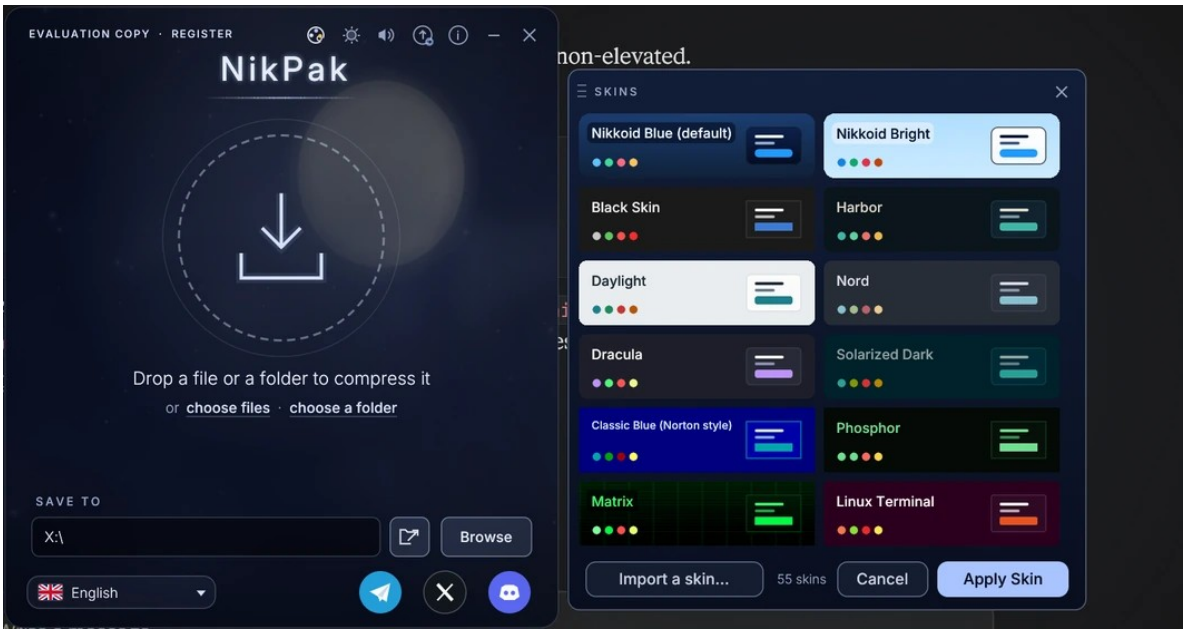


unNik browsing an archive: folders on the left, files in the middle, the preview on the right.

unNik is free for everyone, for ever, with no registration — it is what makes a `.nik` archive safe to send: the recipient needs nothing but this. It opens an archive in a window, browses folders, searches by name, views text, images and the cover, and extracts.

- **Browse before extracting** — nothing is written to disk until you ask for it.
- **Search** across the archive's file names, and sort by name, size, date or type.
- **View** text and images from inside the archive without unpacking them.
- **Extract** everything, a folder, or the selection, to a folder you choose.
- **Ask for the password** only for archives that have one; the rest open silently.

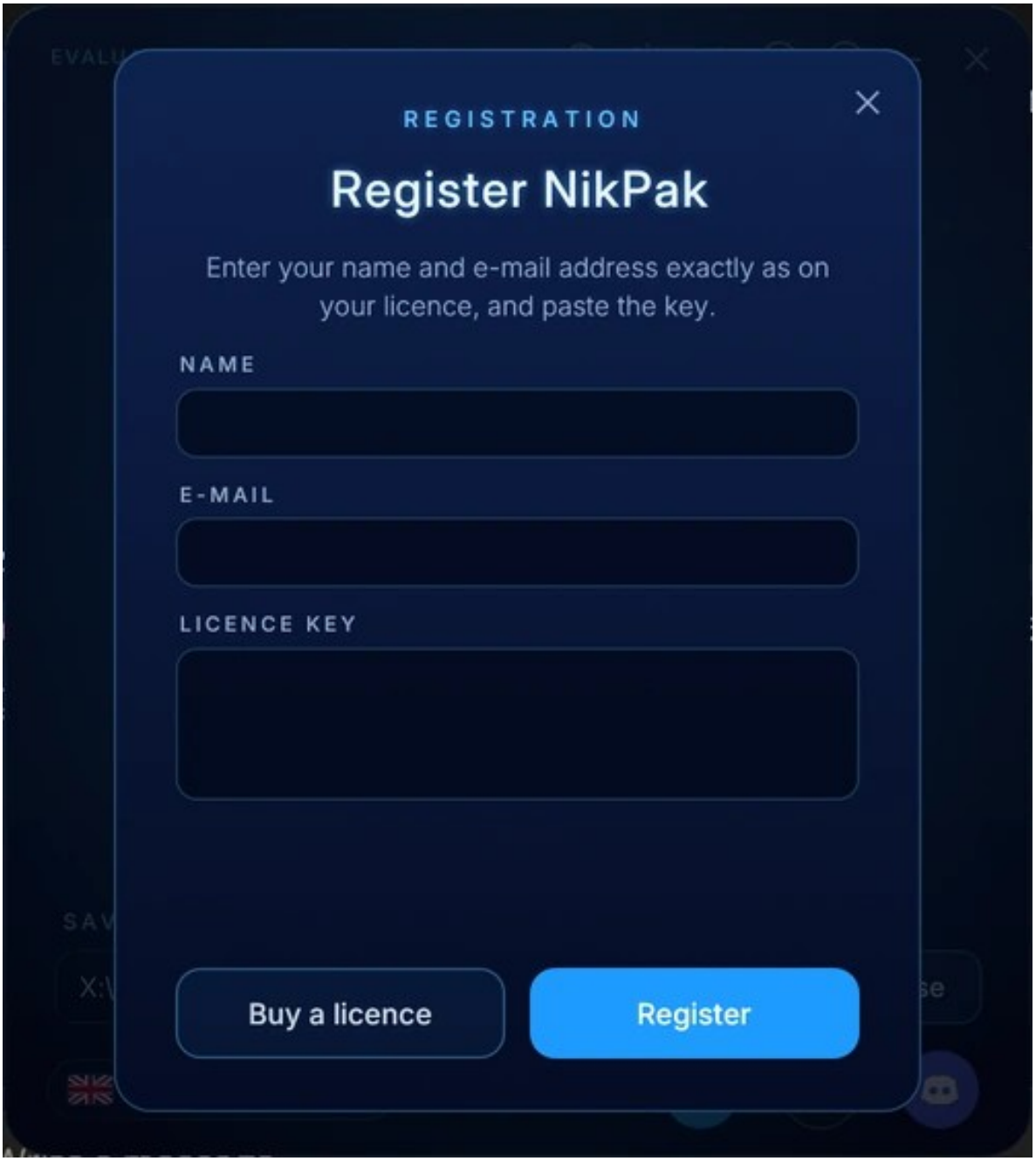
8. Skins, languages and registration



The skin box beside the window: 89 skins, chosen with a live preview.

89 skins, from Amiga to ZX Spectrum, with a live preview beside the window. Ten interface languages — English, 中文, हिन्दी, Español, العربية, Français, □□□□□, Português, Русский and Bahasa Indonesia — chosen with the flag button.

nikpak is shareware: free for individuals, licensed per seat for companies. *Help* ▸ *Register* takes the name, e-mail and key exactly as on the licence and checks it **offline**. unNik needs no licence at all.



The registration box.

Keys you will use

Key	Action
`F1 · F10`	Help · skins
`Ctrl+O · Ctrl+N`	Open an archive · new archive
`Ctrl+E`	Extract
`Ctrl+T`	Test the archive
`F5`	Refresh
`Ctrl+Shift+L`	Interface language

Part II · Technical guide

How it is built, what it stores, and how to rebuild it.

1. Architecture

NikPak is a Rust workspace of two binaries over one library: `nikpak` (the archiver, window and command line) and `unNik` (the browser). The library owns the format, the codecs and the crypto, so the two programs cannot disagree about what an archive means.

The pieces

Part	What it does
`The container`	Header, block index, per-block compressor, the encrypted name table
`The compressors`	Zstandard (fast levels), LZMA2 and PPMd (maximum), per block
`The Deflate recompressor`	Decodes the inner streams of Office files, PDFs and gzip, recompresses the plain content, rebuilds them bit-exactly
`The media filters`	WAV, AIFF, MOD and BMP trial compression, smallest result kept
`The NAC audio codec`	A lossless audio path for the media filters
`The crypto`	Argon2id key derivation, XChaCha20-Poly1305, authenticated headers
`unNik`	The same library, read-only, with the browser UI

The GUI is egui, which is why the window is one executable with no runtime to install: the archiver and the browser ship as single files, and the self-extracting `.exe` is unNik with the archive appended.

2. The .nik format

An archive is a **header**, a **block index**, the **blocks** and an **encrypted name table**. The header carries the format version, the flags, the Argon2id parameters and the salt; the index lists every block with its compressor, its offset and its compressed and uncompressed sizes.

- **Blocks are independent.** That is what makes multi-threaded compression possible, and it is what makes a damaged block cost one file rather than the archive.
- **Names are in their own table**, encrypted with the content, so an archive leaks neither the content nor the shape of what is in it.
- **Authenticated.** The header and the index are covered by the same key, so a tampered index is detected before anything is written.
- **Versioned.** A newer format is refused with a message rather than read partially.
- **Volumes** are the same stream cut at block boundaries, so any volume can be the one you open, and the rest are found by name.

3. The compression engine

The engine chooses a compressor **per block** rather than per archive, because the data inside one folder rarely has one character: a photograph and a source file in the same archive should not be forced through the same path. At the maximum level binary data

goes to LZMA2 and text to PPMd, and the choice is recorded in the block index so extraction needs no guess.

Deflate recompression is the other half of the size win. Office documents, PDFs and `.gz` files are Deflate streams inside a container; NikPak decodes the inner stream, compresses the plain content with a stronger algorithm, and on extraction rebuilds the inner Deflate stream **byte for byte**, including its block boundaries — so the file that comes out is identical to the one that went in, and the archive is much smaller than one that stored it.

> The media filters work by trial: WAV, AIFF, MOD and BMP are compressed with the general path and with a purpose-built filter (adaptive prediction, byte-plane shuffling, a pixel predictor), and the **smallest** result is kept. The filter costs a second pass and buys 12–15 % on audio and up to 38.8 % against RAR 7 on photographs.

4. Encryption, and why it is quantum-resistant

A password goes through **Argon2id** — memory-hard, so a GPU or an ASIC gains little — and the resulting key encrypts with **XChaCha20-Poly1305**, an authenticated stream cipher with a 192-bit nonce, which is wide enough to pick a random nonce per archive without collision.

- **256-bit symmetric keys only.** There is no RSA and no elliptic curve in the format, so there is no public-key exchange for a future quantum computer to break: the security rests on the symmetric cipher and on the password.
- **Authenticated, not just encrypted.** Poly1305 covers the content; the header and the index are authenticated by the same key, so a modified archive fails rather than decoding to something else.
- **File names encrypted.** The name table is not a plain list — an archive does not reveal what it contains.
- **No recovery path**, by design: nothing in the program can decrypt an archive without its password.

> The claim is deliberately narrow: **the format has no public-key step to break**. A weak password is still a weak password, which is why the key derivation is memory-hard rather than a plain hash.

5. Benchmarks, hardening and building

Against RAR, 7-Zip and ZIP

What the tests measured

Corpus	Against RAR 7.00	Against 7-Zip 23.01	Against ZIP
`Photographs`	38.8 % smaller	ahead	far ahead
`Audio (WAV, AIFF)`	12–15 % smaller	ahead	far ahead

Corpus	Against RAR 7.00	Against 7-Zip 23.01	Against ZIP
Office documents and PDFs	ahead, thanks to Deflate recompression	ahead	far ahead
Mixed folders	ahead at maximum	level with at high	far ahead
Speed at the fast levels	faster	level	slower than Store, by design

The numbers are from the published test corpus and the scripts are in the repository, so they can be re-run rather than believed. The honest summary: at maximum compression NikPak wins on media and on Office files, is level with 7-Zip on mixed data, and is not trying to win a speed race at the fast levels.

Hardening

- **Bounds-checked parsing.** Every length in the file is validated against the remaining bytes before it is used; a malformed archive is an error, not a crash.
- **Extraction is confined** to the target folder: an entry named `..\..\windows\system32` is refused, not written.
- **No unsafe code in the parsers** where it can be avoided, and where it cannot, the invariants are documented beside the block.
- **The tests include malformed archives** — truncated headers, wrong checksums, a block index that points past the end — because a format that only ever sees valid files is a format that has never been tested.

Building

```
# Windows
.\build.ps1              # dist\nikpak.exe and dist\unNik.exe
.\build.ps1 -Test        # run the tests first
.\build.ps1 -Installer   # also the setup (needs Inno Setup 6)

# macOS / Linux
./build.sh               # dist/nikpak and dist/unNik

# command-line only, no window
cargo build --release --no-default-features
```

The release scripts keep **one version in one place**: `buildwin.ps1` writes the new number into `Cargo.toml`, `Cargo.lock`, `website/site.json` and `website/version.json` before compiling and restores it if the build fails; `buildmac.ps1` and `buildunix.ps1` read the same `Cargo.toml` and refuse to build when the pushed source carries a different number.

Each platform's script ends by writing `NikPak-<version>-<Platform>.zip` with the installer, `unNik`, a `README.txt` and the eleven manuals in a `Documentation` folder, so the site offers one download per platform and nothing loose.

6. This book

The eleven PDFs are generated from `documents/text/<code>.py` by `documents/build_docs.py`, with the same typesetting engine as the other Nikkoid products (`..\nkdocs\nkdocs.py`). Run `python documents/build_docs.py` after changing the text; `scripts\build-docs.ps1` does it before the zips are packed.